Doubling Release Frequency with Zero-Downtime Deploys

GitHub Actions CI/CD, Docker Containers, Terraform Infrastructure as Code and Automated Test Gates

A simple-language guide to the project: summary, components, workflow and architecture

> **The project in one sentence (as written)**
> Doubled release frequency with zero-downtime deploys by building CI/CD automation (GitHub Actions), Docker containerization, Terraform IaC, and automated unit, integration, and regression test frameworks gating every change.

Contents

1. Overall summary (in simple terms)
2. Decoding the project sentence, phrase by phrase
3. Key ideas explained simply (a mini-glossary)
4. Architecture diagrams and a full walk-through of them
5. Components explained in detail
6. Workflows explained end to end
7. How release frequency doubled
8. Risks and safeguards
9. Quick recap

Note: the project sentence is a short summary. This guide describes a realistic, typical design for such a project so that every part of the sentence can be explained concretely. Component names, timings, release counts and code snippets are illustrative examples chosen for this guide, not data from a specific system.

1. Overall Summary (in Simple Terms)

Think of a busy **restaurant** that wants to change its menu often without ever closing its doors. Before this project, releasing new software was like closing the restaurant for an evening: a few people followed a long manual checklist, copied files onto servers by hand, hoped nothing broke, and customers saw an "under maintenance" page. Because each release was scary and slow, the team released rarely, so each release contained lots of changes, which made it even riskier.

The project replaced that with an **automated assembly line**:

- **GitHub Actions (CI/CD):** every code change automatically triggers a pipeline that builds, tests and deploys it. No manual copying.
- **Docker:** the application is packed into a **container image**, a sealed lunchbox containing the app and everything it needs. The exact same box is tested and then shipped to production, so "it worked on my machine" problems disappear.
- **Terraform (Infrastructure as Code):** servers, networks, load balancers and databases are described in code files. Environments are created the same way every time, reviewed like code, and never hand-configured.
- **Automated tests as gates:** unit, integration and regression tests run on every change. If any fail, the change stops; it cannot reach users.
- **Zero-downtime deploys:** new versions start next to the old one, are checked, and then traffic is moved over gradually. If something looks wrong, traffic flips back instantly. Users never see an outage.

Because releasing became safe, fast and boring, the team could release smaller changes more often: **release frequency doubled** (for example from about 4 to about 8 production releases per month) while users experienced **no downtime** during releases.

Before	After
Manual release checklist, evenings or weekends	One automated pipeline triggered by merging a pull request
Servers configured by hand; staging differs from production	Terraform creates staging and production from the same code
App installed directly on servers with different library versions	One Docker image, tagged by commit, used in every environment
Tests run sometimes, by hand, if there was time	Unit, integration and regression tests run automatically and block bad changes
Maintenance window and downtime during deploys	Blue-green / rolling deploys with health checks and instant rollback
Big, risky releases every couple of weeks	Small, low-risk releases, twice as often

In one breath

Developer opens a pull request → GitHub Actions runs lint and unit tests → builds one Docker image → runs integration tests on real containers → publishes the image → deploys to a Terraform-built staging environment and runs regression tests → deploys the same image to production by starting it next to the old version and shifting traffic gradually → monitoring rolls back automatically if anything goes wrong.

2. Decoding the Project Sentence, Phrase by Phrase

Phrase	What it means in simple words
Doubled release frequency	Twice as many releases to production in the same period of time.
Zero-downtime deploys	Releasing a new version without any moment where users cannot use the service.
CI/CD automation	Continuous Integration (automatically building and testing every change) and Continuous Delivery/Deployment (automatically releasing changes that pass).
GitHub Actions	GitHub's built-in automation service that runs workflows (pipelines) when code is pushed, a pull request is opened, or on a schedule.
Docker containerization	Packaging the application and its dependencies into a standard image that runs the same everywhere as a container.
Terraform IaC	Describing cloud infrastructure in code files, which Terraform reads to create, change or remove real resources.
Automated unit tests	Small, fast tests that check individual functions or classes in isolation.
Integration tests	Tests that check several real parts working together, such as the app with a real database and queue.
Regression tests	A broad suite checking that existing features still work after a change, run against a deployed environment.
Test frameworks	The tools and shared helpers that make writing and running these tests easy and consistent.
Gating every change	No change can merge or deploy unless the required checks pass.

3. Key Ideas Explained Simply (a Mini-Glossary)

3.1 CI and CD

- **Continuous Integration (CI):** developers merge small changes often; each change is automatically built and tested so problems are found within minutes.
- **Continuous Delivery:** every change that passes is ready to release with one click or approval.
- **Continuous Deployment:** every change that passes is released automatically.
- **Pipeline:** the ordered list of automated steps a change goes through.

3.2 GitHub Actions vocabulary

- **Workflow:** a YAML file in `.github/workflows/` describing an automated process.
- **Trigger (event):** what starts it, such as `pull_request`, `push` to main, or a schedule.
- **Job:** a group of steps that runs on one machine; jobs can run in parallel or depend on each other (`needs`).
- **Step / action:** a single command or a reusable building block (for example `actions/checkout`).
- **Runner:** the machine that executes the job (GitHub-hosted or self-hosted).
- **Environments:** named targets such as staging and production, which can require approvals and hold their own secrets.
- **OIDC:** lets a workflow get short-lived cloud credentials without storing long-lived cloud keys in GitHub.
- **Branch protection / required checks:** rules that stop merging into main until chosen checks pass and reviewers approve.

3.3 Docker vocabulary

- **Dockerfile:** the recipe for building an image.
- **Image:** the sealed, read-only package (app + runtime + libraries). **Container:** a running instance of an image.
- **Layers and caching:** each recipe step makes a layer; unchanged layers are reused, so builds are fast.
- **Multi-stage build:** compile in a big “builder” image, then copy only the result into a small runtime image (smaller, safer, faster to start).
- **Registry:** a store for images (for example GitHub Container Registry, Amazon ECR).
- **Immutable tag:** tagging each image with the commit SHA so a tag always means exactly one build.

3.4 Terraform vocabulary

- **Provider:** a plugin that talks to a platform (AWS, Azure, GCP, Kubernetes).
- **Resource:** one piece of infrastructure, such as a load balancer or database.

- **Module:** a reusable group of resources (for example “service with load balancer”).
- **Plan:** a preview of exactly what Terraform would create, change or destroy.
- **Apply:** carrying out the plan.
- **State:** Terraform’s record of what it manages, stored remotely and shared; a **lock** ensures only one apply runs at a time.
- **Drift:** when real infrastructure no longer matches the code (for example someone changed something by hand).

3.5 Test types and gates

Tests are often drawn as a **pyramid**: many fast unit tests at the bottom, fewer integration tests in the middle, and a smaller number of slower end-to-end/regression tests at the top.

Test type	What it checks	Speed	Where it runs
Unit	One function or class, with fakes for databases and networks	Milliseconds each	Every pull request (stage 2)
Integration	App + real database + real queue in containers	Seconds each	Every pull request (stage 4)
Regression	Whole user journeys and API contracts on a deployed environment	Minutes for the suite	Staging after each merge (stage 6)
Smoke	A few critical paths right after a deploy	Seconds	On the new version before and during traffic shift

3.6 Zero-downtime deployment strategies

- **Rolling:** replace instances a few at a time; the rest keep serving.
- **Blue-green:** run the new version (green) next to the old one (blue), test it, then switch traffic. Blue stays ready for instant rollback.
- **Canary:** send a small slice of traffic (for example 10%) to the new version first, compare, then increase.
- **Readiness check:** “is this instance ready to receive traffic?” The load balancer only sends traffic to ready instances.
- **Liveness check:** “is this instance still working?” Unhealthy instances are restarted.
- **Graceful shutdown / connection draining:** old instances stop taking new requests but finish the ones in progress before shutting down.
- **Backward-compatible database changes (expand and contract):** add new columns or tables first, deploy code that works with both old and new, and remove old ones only later, so old and new versions can run at the same time.

3.7 Measuring delivery (DORA metrics)

- **Deployment frequency:** how often you release (this project doubled it).
- **Lead time for changes:** time from commit to production.
- **Change failure rate:** share of releases that cause a problem.
- **Time to restore:** how quickly you recover from a bad release.

4. Architecture Diagrams and Full Walk-Through

Figure 1 shows the whole system: the GitHub Actions pipeline on the left and the Terraform-managed infrastructure and environments on the right. **Figure 2** zooms into stage 7 and shows, step by step, how a new version replaces the old one without downtime.

How to read Figure 1	Meaning
Blue zone (left)	The GitHub Actions pipeline. Stages 1 to 7 run in order for every change.
Red boxes	Test gates: a failure here stops the pipeline.
Orange zone (right), purple boxes	Infrastructure as code (Terraform code, plan/apply, state) and test reports.
Orange zone (right), orange boxes	Where images and running software live: registry, staging, production.
Green boxes	The developer, the cloud account, and users.
Dark blue arrows 1–8	The order of pipeline stages, ending with monitoring after deploy (8).
Brown arrows D1–D3, PL	Publishing the image, deploying it to staging and production, and environments pulling it.
Purple arrows I, P1–P4, R	Infrastructure changes flowing through Terraform into staging and production; test results to reports.
Green arrow U	Users' traffic reaching production, uninterrupted.

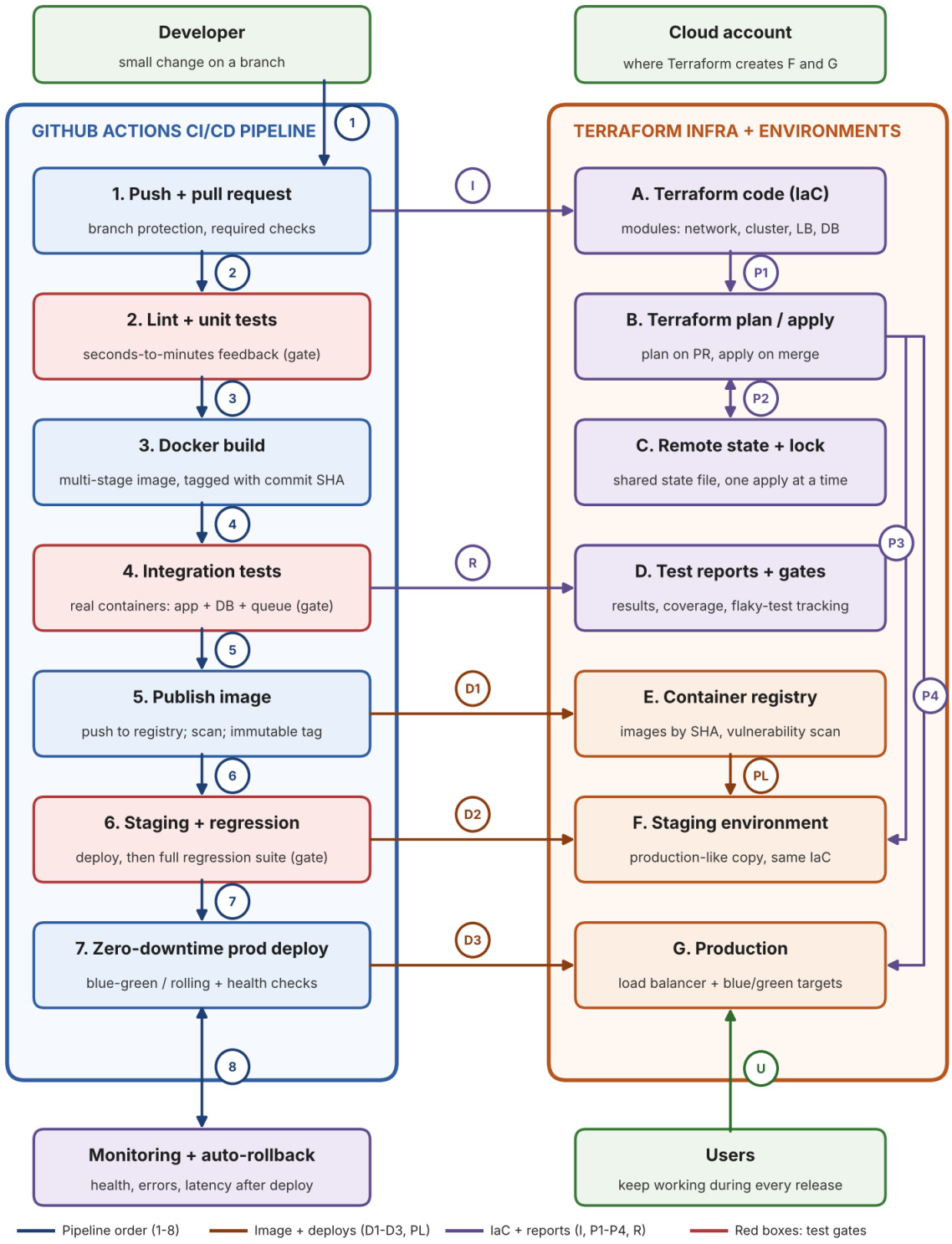


Figure 1. CI/CD architecture: GitHub Actions pipeline with test gates, Docker registry, Terraform-managed staging and production

4.1 The zones of Figure 1

Blue zone – GitHub Actions pipeline. Each box is a job (or group of jobs) in GitHub Actions workflows. The red boxes are the automated test gates the project built. The order goes from cheapest and fastest checks to slowest, so most problems are caught within minutes.

Orange zone – Terraform infrastructure and environments. The purple boxes are the infrastructure-as-code machinery and the test-report store. The orange boxes are the registry holding images and the two environments (staging and production) that Terraform creates in the cloud account.

Green boxes. The developer starts everything; users are the people who must never notice a release; the cloud account is where Terraform builds the environments.

4.2 Every box in Figure 1

Box	Zone	One-line explanation
Developer	External	Makes a small change on a branch and opens a pull request.
Cloud account	External	The cloud (for example AWS) where Terraform creates staging and production.
1. Push + pull request	Pipeline	Triggers workflows; branch protection requires checks and review before merge.
2. Lint + unit tests	Pipeline (gate)	Code style, static checks and fast unit tests.
3. Docker build	Pipeline	Builds one multi-stage image tagged with the commit SHA.
4. Integration tests	Pipeline (gate)	Runs the image with real dependencies in containers and tests them together.
5. Publish image	Pipeline	Scans and pushes the image to the registry with an immutable tag.
6. Staging + regression	Pipeline (gate)	Deploys to staging and runs the full regression suite.
7. Zero-downtime prod deploy	Pipeline	Deploys the same image to production using blue-green or rolling with health checks.
A. Terraform code (IaC)	Infra	Version-controlled modules describing network, compute, load balancer and database.
B. Terraform plan / apply	Infra	Shows a plan on pull requests and applies it on merge, run by GitHub Actions.

Box	Zone	One-line explanation
C. Remote state + lock	Infra	Shared state file and lock so applies are consistent and never run twice at once.
D. Test reports + gates	Reports	Stores results, coverage and flaky-test data; feeds required status checks.
E. Container registry	Runtime	Holds images by commit SHA; scanned for vulnerabilities.
F. Staging environment	Runtime	A production-like copy built from the same Terraform modules.
G. Production	Runtime	Load balancer plus blue and green target groups (or a rolling deployment).
Monitoring + auto-rollback	Ops	Watches health, errors and latency after deploy and triggers rollback.
Users	External	Keep using the service throughout every release.

4.3 Every arrow in Figure 1

Arrow	From → To	What travels along it
1	Developer → Push + PR	A branch push and a pull request trigger the workflows.
2	PR → Lint + unit tests	Source code checked out on a runner.
3	Unit tests → Docker build	Only if lint and unit tests pass.
4	Docker build → Integration tests	The freshly built image, started with its dependencies.
5	Integration tests → Publish image	Only if integration tests pass (and, for deploys, after merge to main).
6	Publish → Staging + regression	The image tag (commit SHA) to deploy.
7	Staging + regression → Prod deploy	Only if the regression suite passes (plus approval if required).
8	Prod deploy ↔ Monitoring	After deploy, health and error metrics are watched; a bad signal triggers rollback.
I	Pipeline → Terraform code	Pull requests that change infrastructure code run the Terraform workflow.

Arrow	From → To	What travels along it
P1	Terraform code → Plan/apply	Code is validated, planned and (after merge) applied.
P2	Plan/apply ↔ Remote state	Reads current state, takes the lock, writes the new state.
P3	Plan/apply → Staging	Creates or updates staging resources.
P4	Plan/apply → Production	Creates or updates production resources (with approval).
R	Integration tests → Test reports	Results and coverage from every test stage (shown from stage 4 as an example).
D1	Publish → Registry	The image is pushed with tag = commit SHA.
PL	Registry → Staging	Staging pulls that exact image (production pulls the same one later).
D2	Staging + regression → Staging env	Deployment of the new image to staging.
D3	Prod deploy → Production	Zero-downtime deployment of the same image to production.
U	Users → Production	Normal traffic, which keeps flowing during the release.

4.4 Why Figure 1 is shaped this way

- **Fast gates first:** linting and unit tests take minutes, so most mistakes are caught before slower steps run.
- **Build once, deploy many:** the image built at stage 3 is the only artefact; staging and production run the same image, so what was tested is what ships.
- **Infrastructure follows the same path as code:** Terraform changes are reviewed in pull requests and applied by the pipeline, never by hand.
- **Staging and production come from the same modules:** staging is a trustworthy rehearsal of production.
- **Monitoring closes the loop:** the deploy is not “done” until health checks confirm it, and it can undo itself.

4.5 Figure 2: how a zero-downtime release works

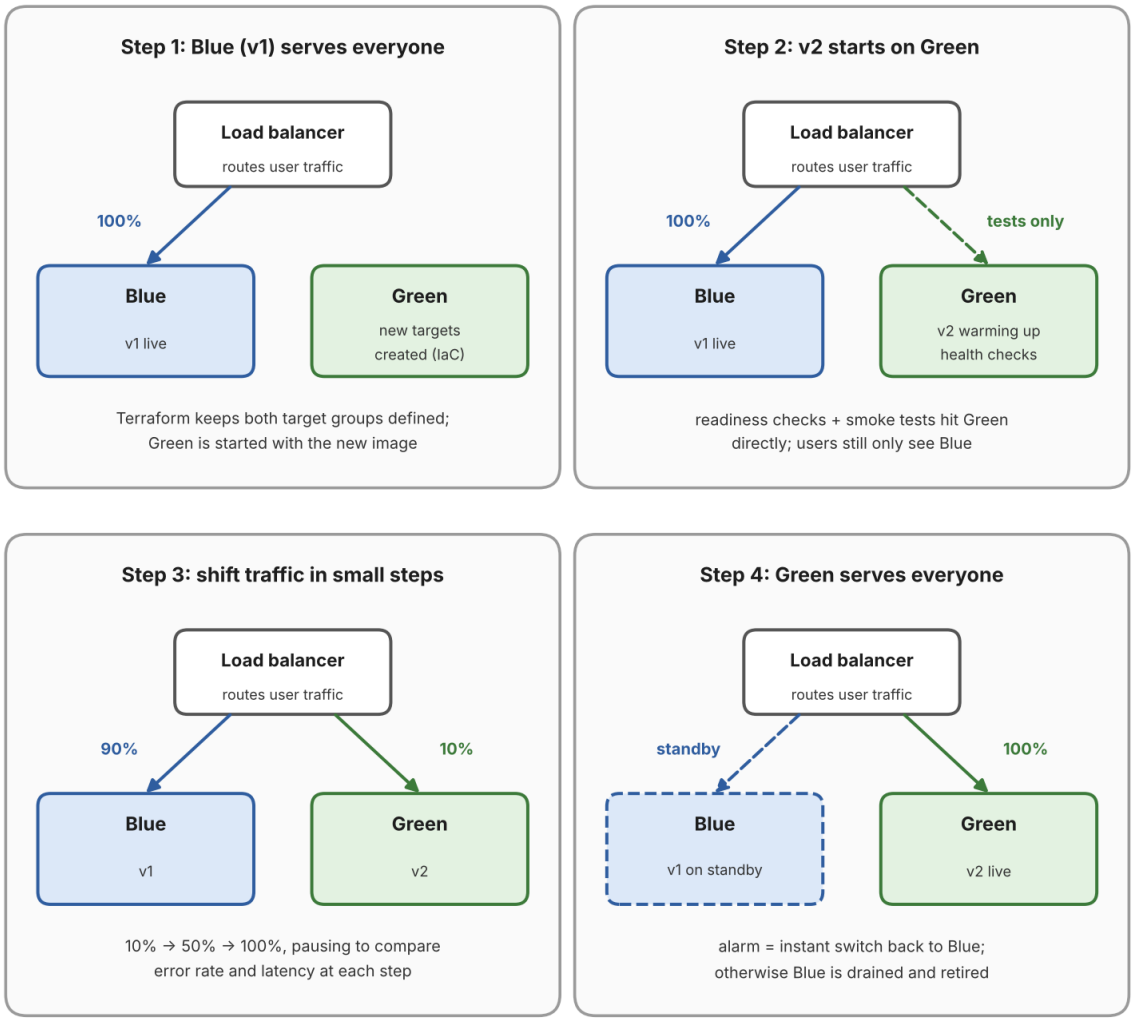


Figure 2. Blue-green release with gradual traffic shifting and instant rollback

Step	What happens	Why users notice nothing
1. Blue serves everyone	Version 1 (blue) handles 100% of traffic. Terraform has defined both blue and green target groups behind the load balancer; the pipeline starts green with the new image.	Nothing has changed for users yet.
2. v2 starts on Green	Green containers start; readiness checks wait until each is warmed up; smoke tests call green directly (dashed arrow: tests only).	The load balancer sends real users only to blue.

Step	What happens	Why users notice nothing
3. Shift traffic in steps	The load balancer sends 10% of traffic to green, then 50%, then 100%, pausing to compare error rates and latency against blue.	Both versions are healthy and compatible; any problem affects only a small slice briefly.
4. Green serves everyone	Green gets 100%. Blue stays running on standby; if an alarm fires, traffic flips back instantly. Otherwise blue is drained (it finishes in-progress requests) and retired.	No requests are dropped: old instances finish what they started before stopping.

Rolling deploys follow the same principles for services that do not use blue-green: replace a few instances at a time, add new ones before removing old ones (`maxUnavailable: 0`), and only continue while health checks pass.

5. Components Explained in Detail

Each component has a fact table followed by a step-by-step description of how it works.

5.1 External parts

Developer

What it is	An engineer making a change: a feature, a fix, a test or an infrastructure update.
Simple analogy	A chef proposing a new dish for the menu.
New habits	Small pull requests, tests written alongside code, risky features hidden behind feature flags, database changes made backward compatible.

How it works, step by step

1. Creates a branch from main and makes a small, focused change.
2. Runs unit tests locally (the same commands the pipeline uses).
3. Pushes the branch and opens a pull request (arrow 1).
4. Reads pipeline results in the pull request and fixes any failures.

Cloud account

What it is	The cloud provider account(s) where infrastructure runs, often separate accounts for staging and production.
Simple analogy	The building where the restaurant's kitchens are installed.
Access rule	No one changes production by hand. Terraform (through the pipeline) is the only writer; people have read-only access plus emergency break-glass access that is logged.

How it works, step by step

1. GitHub Actions authenticates with OIDC and assumes a narrowly scoped role.
2. Terraform creates or updates resources inside the account.
3. Audit logs record every change.

5.2 GitHub Actions pipeline

1. Push + pull request

What it is	The trigger for the pipeline and the place where review happens.
Simple analogy	Handing a new recipe to the head chef for tasting before it goes on the menu.
Branch protection	Main cannot be pushed to directly; required status checks (stages 2 and 4, plus Terraform plan when infra changes) must pass; at least one reviewer must approve; branches must be up to date.

How it works, step by step

1. The push triggers workflows defined in `.github/workflows/`.
2. Path filters decide which jobs run (for example Terraform jobs only when `infra/` changes).
3. Concurrency rules cancel outdated runs when new commits are pushed to the same pull request.
4. Results appear as checks on the pull request; merging is blocked until they are green and a reviewer approves.

2. Lint + unit tests (gate)

What it is	The first and fastest gate.
Simple analogy	Checking each ingredient before cooking.
Unit test framework	A shared setup per language (for example pytest, JUnit, Jest) with helpers for fakes and test data, a coverage threshold for changed code, and parallel test execution.
Target time	A few minutes, so developers get feedback quickly.

How it works, step by step

1. Checks out the code and restores cached dependencies.
2. Runs linters and formatters (style and simple bug patterns) and type checks.
3. Runs unit tests in parallel shards.
4. Uploads results and coverage to test reports (D).
5. Fails the check if any test fails or coverage of new code drops below the threshold.

3. Docker build

What it is	Builds the application image from the Dockerfile.
Simple analogy	Packing the finished dish into a sealed, labelled lunchbox.
Good practices	Multi-stage builds; small base images; pinned base image versions; dependency layers before source code for good caching; runs as a non-root user; includes a health-check endpoint; image tagged with the commit SHA (and never relying on `latest`).

How it works, step by step

1. Uses Docker Buildx with layer cache stored between runs, so unchanged layers are reused.
2. Builder stage installs build tools and compiles the app.
3. Runtime stage copies only the compiled app and runtime libraries.
4. Tags the image `app:<commit-sha>` and adds labels (source repository, commit, build time).
5. Passes the image to integration tests (arrow 4).

Illustrative multi-stage Dockerfile:

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
USER node
EXPOSE 8080
CMD ["node", "dist/server.js"]
```

4. Integration tests (gate)

What it is	Tests the real image together with real dependencies.
Simple analogy	Cooking the full dish in a test kitchen with the real oven and real ingredients.
Integration test framework	Docker Compose or Testcontainers starts the app image, a database, a cache and a message queue; migrations run on the test database; test data is created by shared fixtures; each test cleans up after itself.

How it works, step by step

1. Starts the dependency containers and waits until they are ready.

2. Applies database migrations to an empty test database (proving migrations work).
3. Starts the app container built in stage 3.
4. Runs tests through the app's real API: saving and reading data, sending messages, handling errors and timeouts.
5. Sends results to test reports (arrow R) and fails the check on any failure.

5. Publish image

What it is	Stores the tested image in the container registry.
Simple analogy	Putting the approved lunchbox in the warehouse with a unique serial number.
When it runs	After merge to main (pull requests build and test images but do not publish them for deployment).

How it works, step by step

1. Scans the image for known vulnerabilities; critical issues fail the job.
2. Logs in to the registry with short-lived credentials.
3. Pushes the image with tag = commit SHA (arrow D1); tags are immutable.
4. Records the tag as the output that later deploy jobs use.

6. Staging deploy + regression tests (gate)

What it is	Deploys the image to staging and runs the full regression suite.
Simple analogy	A full dress rehearsal of dinner service before opening the doors.
Regression test framework	End-to-end and API contract tests covering the important user journeys (login, search, checkout, reports), run in parallel against staging, with stable test accounts and data reset between runs. Flaky tests are quarantined and tracked, not ignored.

How it works, step by step

1. Deploys the new image to staging (arrow D2) using the same zero-downtime method as production; staging pulls it from the registry (arrow PL).
2. Waits for readiness checks; runs smoke tests.
3. Runs the regression suite in parallel.
4. Publishes results to test reports; any failure blocks promotion to production.

7. Zero-downtime production deploy

What it is	The job that releases the image to production without interrupting users (Figure 2).
Simple analogy	Opening a second kitchen with the new menu, sending a few guests there first, then everyone, while the old kitchen stays ready.
Controls	GitHub Environment `production` (optional approval, restricted to main); concurrency group so only one production deploy runs at a time; automated rollback on failed health checks or alarms.

How it works, step by step

1. Starts the new version (green) with the same image tag that passed staging (arrow D3).
2. Waits for readiness checks on every new instance; runs smoke tests directly against green.
3. Shifts traffic gradually (10% → 50% → 100%), checking error rate and latency at each step.
4. Keeps blue on standby; drains and removes it after a safe period.
5. Records the release (version, time, who merged) for traceability.

Monitoring + auto-rollback (step 8)

What it is	Metrics, logs and alarms watched during and after every deploy.
Simple analogy	The head waiter watching guests' faces after a new dish is served.
Signals	HTTP error rate, latency percentiles, failed health checks, crash loops, key business metrics (orders per minute).

How it works, step by step

1. Compares the new version with the old during traffic shifting.
2. If a threshold is crossed, triggers rollback: traffic returns to blue (blue-green) or the previous image is redeployed (rolling).
3. Notifies the team with links to the release and logs.
4. After a stable period, marks the release healthy.

5.3 Terraform infrastructure, reports and environments

A. Terraform code (IaC)

What it is	Code files describing every piece of infrastructure, organised into reusable modules.
Simple analogy	Architect's blueprints from which identical kitchens can be built.
Structure	Modules for network, container cluster/service, load balancer with blue and green target groups, database, DNS; one folder per environment that uses the same modules with different sizes and settings.

How it works, step by step

1. An engineer changes a module or an environment setting in a pull request (arrow I).
2. Formatting and validation run automatically.
3. Security and policy checks (for example no public databases) run on the code.
4. The change is reviewed alongside the Terraform plan.

Illustrative Terraform snippet:

```
module "api_service" {
  source         = "../modules/service"
  name           = "api"
  image          = "registry.example.com/api:${var.image_sha}"
  desired_count  = 6
  health_path    = "/health/ready"
  deploy_style   = "blue_green"
}
```

B. Terraform plan / apply

What it is	The GitHub Actions workflow that runs Terraform.
Simple analogy	Showing the builders' exact list of changes before any wall is moved.
Rules	Plan on every pull request and post it as a comment; apply only after merge, using the saved plan; production apply requires environment approval; credentials via OIDC.

How it works, step by step

1. Runs `terraform init` (connects to remote state) and `terraform plan` (arrow P1).
2. Posts the plan to the pull request so reviewers see exactly what will change.
3. After merge, takes the state lock and runs `terraform apply` on the reviewed plan (arrow P2).
4. Applies to staging first (arrow P3), then production after approval (arrow P4).

5. A scheduled job runs `plan` nightly to detect drift and alerts if reality differs from code.

C. Remote state + lock

What it is	The shared record of what Terraform manages (for example in an S3 bucket with a lock table, or Terraform Cloud).
Simple analogy	The single, up-to-date register of what has been built, with a "builder at work" sign on the door.
Protections	Encrypted, versioned, access-restricted; locking prevents two applies corrupting state.

How it works, step by step

1. Terraform reads the state to compare with the code.
2. Before apply, it acquires the lock; other runs wait.
3. After apply, it writes the new state and releases the lock.
4. Older versions are kept so mistakes can be recovered.

D. Test reports + gates

What it is	Where all test results, coverage and timings are collected.
Simple analogy	The kitchen's quality log.
Uses	Required status checks on pull requests; trend dashboards; flaky test detection (tests that pass and fail without code changes) so they can be fixed or quarantined.

How it works, step by step

1. Receives JUnit-style results and coverage from stages 2, 4 and 6 (arrow R).
2. Shows failures directly in the pull request with logs.
3. Tracks test duration so slow tests can be optimised and the pipeline stays fast.

E. Container registry

What it is	The store of built images.
Simple analogy	The warehouse of sealed, numbered lunchboxes.
Rules	Immutable tags; retention policy deletes old unused images; vulnerability scanning on push and periodically; only the pipeline can push.

How it works, step by step

1. Receives the image from stage 5 (arrow D1).
2. Serves the exact image to staging (arrow PL) and later production.
3. Keeps previous images so rollback to an older version is instant.

F. Staging environment

What it is	A production-like environment built from the same Terraform modules.
Simple analogy	A rehearsal kitchen identical to the real one.
Differences from production	Smaller sizes, test data only, separate accounts and secrets; same architecture, same deploy method, same image.

How it works, step by step

1. Receives infrastructure from Terraform (arrow P3) and the image from the registry.
2. Runs regression tests from stage 6.
3. Proves that the deployment itself (not just the code) works before production.

G. Production

What it is	The live environment users depend on.
Simple analogy	The real dining room, open all the time.
Key parts	Load balancer with blue and green target groups (or rolling deployment), auto-scaling container service, readiness and liveness checks, graceful shutdown with connection draining, database shared by both versions.

How it works, step by step

1. Receives infrastructure from Terraform (arrow P4).
2. Receives new versions from stage 7 (arrow D3) following Figure 2.
3. Serves users continuously (arrow U).
4. Reports metrics to monitoring (step 8).

Users

What it is	Everyone using the product.
What changed for them	No maintenance windows, no error pages during releases, and new features and fixes arrive faster and in smaller pieces.

How it works, step by step

1. Send requests to the load balancer at any time.
2. Are routed only to healthy, ready instances.
3. Experience at most a tiny canary slice of a bad version, briefly, before automatic rollback.

6. Workflows Explained End to End

6.1 One change from idea to production

Step	Where	What happens	Typical time
1	Developer	Small change on a branch; pull request opened.	–
2	Stage 2	Lint, type checks and unit tests run in parallel.	3–5 min
3	Stage 3	Docker image built with cache.	2–4 min
4	Stage 4	Integration tests on real containers.	5–8 min
5	Review	Teammate reviews code (and Terraform plan if infra changed); merges.	varies
6	Stage 5	Image scanned and published with the commit SHA.	1–2 min
7	Stage 6	Deploy to staging; regression suite runs.	10–15 min
8	Stage 7	Zero-downtime production deploy with gradual traffic shift.	10–20 min
9	Monitoring	Watch period; automatic rollback if needed.	15–30 min

6.2 Pull request workflow

1. Developer opens a pull request; GitHub triggers the `ci.yml` workflow.
2. Jobs run in parallel where possible (lint, unit tests, Docker build), then integration tests after the build.
3. If infrastructure files changed, the Terraform workflow posts a plan comment.
4. Results appear as checks; any red check blocks merge.
5. The developer fixes problems and pushes; outdated runs are cancelled automatically.
6. A reviewer approves; the branch is merged into main.

Illustrative GitHub Actions workflow (simplified):

```
on:
  pull_request:
  push:
    branches: [main]
jobs:
  unit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: make lint test-unit
  build:
    needs: unit
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t app:${{ github.sha }} .
      - run: make test-integration IMAGE=app:${{ github.sha }}
  deploy:
    if: github.ref == 'refs/heads/main'
    needs: build
    environment: production
    concurrency: production
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy.sh staging ${{ github.sha }} && make test-regression
      - run: ./deploy.sh production ${{ github.sha }} --strategy blue-green
```

6.3 Docker build and publish workflow

1. Restore the layer cache from previous builds.
2. Build the builder stage (dependencies then source code), then the small runtime stage.
3. Tag the image with the commit SHA and add metadata labels.
4. Run integration tests against this exact image.
5. After merge: scan for vulnerabilities, then push to the registry.
6. Every later step refers to the image only by its SHA tag, never rebuilding it.

6.4 Test gate workflow

1. **Unit gate (stage 2):** all unit tests and lint must pass; new code must meet the coverage threshold.
2. **Integration gate (stage 4):** the real image must work with real dependencies and migrations.
3. **Regression gate (stage 6):** the deployed staging environment must pass all critical user journeys.
4. **Smoke checks (stage 7):** the new production version must pass health and smoke checks before and during traffic shift.
5. Any failure stops the pipeline at that point, marks the check red, and notifies the author.
6. Flaky tests are detected from history, quarantined into a separate non-blocking job with a ticket, and fixed quickly so the gates stay trusted.

6.5 Infrastructure change workflow (Terraform)

1. Engineer changes Terraform code in a pull request (for example adds a cache or changes instance size).
2. Pipeline runs `terraform fmt`, `validate`, policy checks and `plan`, and posts the plan.
3. Reviewer checks the plan: what will be created, changed or destroyed.
4. After merge, the pipeline takes the state lock and applies to staging.
5. Staging tests run; then production apply waits for environment approval.
6. Production is updated; state is written; lock released.
7. Nightly drift detection alerts if anyone changed infrastructure outside Terraform.

6.6 Zero-downtime deploy workflow (detailed)

1. Check that no other production deploy is running (concurrency group).
2. Start green instances with the new image tag; blue keeps serving 100%.
3. Each green instance starts, connects to dependencies and only then reports ready.
4. Run smoke tests directly against green.
5. Shift 10% of traffic to green; compare error rate and latency with blue for a few minutes.
6. Shift to 50%, compare again; then 100%.
7. Keep blue running on standby for a safety window.
8. Drain blue: stop sending new requests, let in-flight requests finish (graceful shutdown), then stop it.
9. Mark the release healthy and record it.

6.7 Database change workflow (expand and contract)

During a blue-green or rolling deploy, old and new versions run at the same time against the same database, so database changes must work for both.

1. **Expand:** release a migration that only adds things (new column or table, nullable or with a default). Old code ignores it.
2. **Migrate code:** release application code that writes to both old and new places and reads from the new one when present.
3. **Backfill:** copy existing data into the new structure in the background.
4. **Switch:** release code that uses only the new structure.
5. **Contract:** in a later release, remove the old column or table once no running version uses it.
6. Migrations run automatically in the pipeline before the new code starts, and were already tested in integration tests and staging.

6.8 Rollback workflow

1. An alarm fires during or after the traffic shift (error rate or latency above threshold, failing health checks).
2. Blue-green: the load balancer immediately sends 100% of traffic back to blue (seconds).
3. Rolling: the pipeline redeploys the previous image tag from the registry.
4. The team is notified with links to the release, logs and metrics.
5. The bad change is fixed or reverted through a normal pull request, and the pipeline runs again.
6. Because database changes are backward compatible, rolling back code never breaks the database.

6.9 Credentials and secrets workflow

1. GitHub Actions requests a short-lived OIDC token for the specific workflow and environment.
2. The cloud provider trusts that token and issues temporary credentials for a narrowly scoped role (staging role or production role).
3. No long-lived cloud keys are stored in GitHub.
4. Application secrets (database passwords, API keys) are stored in the cloud secrets manager and injected at runtime, not baked into images.

6.10 Urgent fix (hotfix) workflow

1. A small fix is made on a branch with a test that reproduces the bug.
2. The same pipeline and gates run; nothing is skipped.
3. Because the pipeline is fast and automated, the fix reaches production in under an hour without manual steps or downtime.

7. How Release Frequency Doubled

The figures below are **illustrative**, showing the typical before/after picture for this kind of project.

Measure	Before	After	Main cause
Production releases per month	about 4	about 8	Automated, low-risk pipeline; smaller changes
Downtime per release	15–30 min maintenance window	None	Blue-green / rolling with health checks and draining
Manual steps per release	about 25	0–1 (approval)	GitHub Actions automation, Terraform
Time from merge to production	days	under 1 hour	Pipeline runs automatically after merge
Environment differences causing failures	common	rare	Same Docker image and same Terraform modules everywhere
Time to recover from a bad release	hours	minutes	Instant traffic switch-back, previous images ready

Why safety and speed went up together

When every change is automatically tested, packaged identically and released without downtime, releasing stops being a risky event. Teams then release smaller changes more often, and smaller changes are easier to test, review and roll back. That positive cycle is what doubled release frequency.

8. Risks and Safeguards

Risk	Why it matters	Safeguard
Slow pipeline	Developers wait and batch changes, reducing frequency.	Parallel jobs, caching, fast tests first, test-duration tracking.
Flaky tests	People stop trusting gates and rerun until green.	Flaky detection, quarantine with tickets, fix quickly.
Incompatible database change	Old and new versions break during the deploy.	Expand-and-contract migrations, tested in integration and staging.
Terraform destroys something important	A mistaken plan could delete resources.	Plan review, `prevent_destroy` on critical resources, approvals, state versioning.
Two deploys at once	Conflicting releases or state corruption.	Concurrency groups in Actions, Terraform state lock.
Leaked cloud credentials	Attackers could change infrastructure.	OIDC short-lived credentials, least-privilege roles, no keys in GitHub.
Vulnerable base image	Known security holes shipped in containers.	Image scanning, pinned and regularly updated base images.
Bad version reaches users	Errors for some users.	Canary traffic steps, alarms, automatic rollback, blue on standby.

9. Quick Recap

Question	Short answer
What was achieved?	Twice as many production releases, with no downtime during releases.
What does GitHub Actions do?	Runs the pipeline on every pull request and merge: tests, build, publish, deploy, Terraform.
What does Docker do?	Packages the app into one image (tagged by commit) that is tested once and run everywhere.
What does Terraform do?	Defines staging and production infrastructure as reviewed code, applied only by the pipeline.
Which tests gate changes?	Unit and integration tests on every pull request; regression tests on staging; smoke checks during production deploys.
How is downtime avoided?	Blue-green or rolling deploys, readiness checks, gradual traffic shifting, graceful draining, backward-compatible database changes, instant rollback.